

Bytes, Not Ratios: What KV Cache Compression Benchmarks Actually Measure

Samjhana Bhusal
samjhana.bhusal.05@gmail.com

September 30, 2026

Abstract

KV cache compression methods for long-context LLM inference are compared, almost universally, at a matched *token* budget—a “compression ratio” that fixes the fraction of cached tokens retained—and the result is reported as a memory saving. We show these are not the same quantity, and that the gap is method-dependent and sometimes total. Auditing the field’s standard toolkit, we find that (i) some widely-benchmarked methods realize head-wise budgets by *masking* rather than eviction, so they report large compression ratios while freeing no memory; (ii) head-level budget allocation cannot reduce memory under grouped-query attention even when implemented physically, because keys and values are shared within a query group; and (iii) peak-memory overheads incurred during compression are invisible to ratio accounting. We introduce a measurement instrument that fixes the budget in *realized bytes* and a wrapper that matches any method to a target byte budget, and re-run the standard comparison. Across Qwen2.5-0.5B and 3B, two of five audited methods report large compression ratios while realizing 100–108% of full-cache bytes: an adaptive head-wise method that masks rather than evicts, and a channel-pruning method whose stored tensors never shrink. The consequence for evaluation is stark: at a nominal 50% “compression” the channel-pruning method attains near *full-cache accuracy on RULER* and appears to dominate, yet the byte correction places it at 100% memory—its advantage was an accounting artifact. Among the methods that genuinely compress, a trivial sink-plus-recency window beats the query-aware scorer at every matched byte budget on RULER. Our results are complementary to concurrent work showing rankings are unstable under query visibility [Luo et al., 2026]: we show they are also unstable under the resource unit the field measures them in. Code and all result files are released.

1 Introduction

KV cache compression papers, and the tooling built to compare them, budget compression in *tokens*: a “compression ratio” fixing the fraction of cached tokens retained, or an absolute token count. NVIDIA’s KVPress toolkit¹, the field’s most widely used comparison harness and the one this paper audits and builds on, exposes exactly this knob. Recent matched-budget audits inherit it: Luo et al. [2026] fix $r \in \{0.25, 0.5, 0.75, 0.9\}$ as a token fraction; Garcia [2026] fix an absolute token count; the earlier cross-family benchmark of Yuan et al. [2024] and the KV-cache-centric SCBench [Li et al., 2025] do the same. In every case the result is reported, implicitly or explicitly, as a statement about *memory*—the resource practitioners actually care about when they compress a cache.

Token fraction and memory fraction are the same quantity only under an assumption the literature rarely states: that every retained token costs the same number of bytes, and that nothing else

¹<https://github.com/NVIDIA/kvpress>

is stored. Neither holds in general. Per-layer and per-head budget allocation (PyramidKV, AdaKV, CAKE, HeadKV) changes how many bytes a “token” costs at different depths; methods that keep per-token bookkeeping (accumulated attention scores, observation-window state) spend bytes the token count does not see; and some implementations realize a head-wise or channel-wise budget by *masking* the attention computation rather than by shrinking the stored tensor at all. This paper is about that last case, because it is the sharpest existence proof that the assumption fails: auditing KVPress’s own **AdaKVPress** by reading its source, we find it returns the key/value tensors *unmodified* and stores a mask for the attention computation to apply downstream (§4). A press that reports a 90% compression ratio can free approximately none of the cache. No token-fraction benchmark can see this, because a token-fraction benchmark never looks at the tensor.

We build a measurement instrument that reads realized bytes directly from the stored cache rather than inferring them from a nominal ratio, use it to classify the standard toolkit’s compression methods into a three-way taxonomy, and re-run the field’s standard comparison with the budget fixed in bytes instead of tokens. The result is not a gentle re-ranking: on RULER, the method that looks best under the field’s token-fraction view is byte-infeasible—it cannot be matched to any budget below full cache at all—and among the methods that do genuinely compress, a method with no query awareness and no learned scoring beats the field’s most widely deployed query-aware method at every matched byte budget.

Contributions.

1. A three-way taxonomy of compression implementations—*memory-faithful*, *simulation-only* (masked), and *incommensurable* (off the token axis)—and a measurement instrument that assigns each method to a class by direct byte accounting (§3).
2. An audit (§4) showing the divergence between reported compression ratio and realized bytes across methods on the standard toolkit, including methods that free zero memory, and a first-principles account of why head-level allocation cannot help under GQA. Reproduced across two model scales (0.5B, 3B).
3. A byte-matched re-ranking (§5) of the standard method set on RULER, quantifying which methods are byte-infeasible and how the standard-benchmark ordering compares once the infeasible methods are removed and the rest are matched on realized bytes.

2 Related Work

KV cache compression for autoregressive inference has grown into a large literature. We group it by mechanism, and note for each family how it interacts with the resource unit — token fraction versus realized bytes — that this paper argues is mis-specified throughout.

Static and heuristic eviction. The earliest training-free methods keep a fixed positional pattern. StreamingLLM Xiao et al. [2024b] retains a few initial “attention sink” tokens plus a recent sliding window; LM-Infinite Han et al. [2024] independently proposes a Λ -shaped mask of the same form. These methods are memory-faithful by construction — what they keep is exactly what they store — and, as recent audits find, they are a far stronger baseline than the literature has credited Luo et al. [2026], Garcia [2026].

Attention-score-based eviction. A second family scores tokens by accumulated attention and evicts the low-scoring ones. H2O Zhang et al. [2023] retains “heavy hitters”; Scissorhands Liu et al. [2023] exploits the persistence of importance across steps; TOVA Oren et al. [2024] drops the token least attended by the most recent query; FastGen Ge et al. [2024] profiles each head and applies a per-head policy; Keyformer Adnan et al. [2024] de-biases the discarded-token distribution with a Gumbel-softmax score. A recurring critique is that attention score is an incomplete importance signal — value-vector norms matter too — and that these methods require the materialized attention matrix, making them incompatible with FlashAttention Park et al. [2025].

Query-aware / observation-window selection. SnapKV Li et al. [2024] introduced the dominant modern idea: pool attention from a short *observation window at the end of the prompt* to decide which prefix tokens to keep. PyramidKV Cai et al. [2024] and PyramidInfer Yang et al. [2024] add a layer-wise budget on top. The observation window is precisely the mechanism that concurrent audits identify as confounded: because long-context benchmarks place the question last, the window overlaps the query, and the method’s apparent skill partly measures query–content overlap rather than intrinsic token importance Luo et al. [2026]. Follow-up work treats the window as a defect to be fixed (Judge Q trains soft query tokens Liu et al. [2025]), reformulates selection to be query-agnostic (KVzip Jang et al. [2025]), or removes the query from the score entirely (KeyDiff Park et al. [2025]).

Adaptive head- and layer-level budget allocation. This family is central to our argument. Rather than a uniform per-head budget, Ada-KV Feng et al. [2025] reallocates a global budget across heads by an L_1 eviction-loss bound; CAKE Qin et al. [2025] allocates across layers by an attention-dispersion preference; HeadKV Fu et al. [2025] scores heads by retrieval-and-reasoning ability; RazorAttention Tang et al. [2025] and DuoAttention Xiao et al. [2025] keep full cache only for identified retrieval/streaming heads. Two facts make this family impossible to price in token fractions. First, under grouped-query attention the unit of KV memory is the (layer, KV-group) cell, not the head: query heads in a group share K and V, so *head-level* budget concentration frees no memory unless the storage layout is made ragged, which requires paged KV management Rehg [2024]. Second, some widely-used implementations realize head-wise budgets by *masking* rather than eviction — the K/V tensors are never shrunk — so the reported compression ratio and the realized byte savings diverge completely. We quantify both effects in §4.

Merging rather than eviction. CaM Zhang et al. [2024b] merges to-be-evicted value caches into survivors; D2O Wan et al. [2025] adds cosine-similarity-weighted token merging under a per-layer dynamic budget; KVMerger Wang et al. [2024] clusters merge candidates by key similarity. Merging changes what a “retained token” means and further loosens the token-count-to-bytes correspondence.

Quantization: a composable, orthogonal axis. KV quantization (KIVI Liu et al. [2024], KVQuant Hooper et al. [2024]) reduces bytes-per-element rather than token count, and composes with eviction rather than competing with it. We do not evaluate quantization, but it is the clearest evidence for our thesis: once precision varies, only a *byte* budget is well defined, and indeed the work nearest to ours in spirit trades token count against bits-per-token and is forced to reason about memory rather than token fraction Zhang et al. [2024a].

Retrieval and offloading. Quest Tang et al. [2024], InfLLM Xiao et al. [2024a], and PQ-Cache Zhang et al. [2025] never permanently discard KV; they page or retrieve it, trading GPU bytes for latency and host memory. These change the resource being budgeted and are out of our eviction-only scope, but they underline that “compression ratio” means different things across families.

Evaluation and critique. The benchmark side is where this paper lives. longctx_bench Yuan et al. [2024] and SCBench Li et al. [2025] compare methods across families under compression ratios; Gao et al. [2025] and Agrawal and Mayer [2026] argue that “compression ratio alone is a poor predictor of end-to-end performance,” and Chen et al. [2026] show that compression damages some prompt content far more than aggregate scores reveal. Most directly, Luo et al. [2026] run a *matched-token* audit and find that rankings invert under query visibility, with SnapKV losing to a start-plus-window baseline. Our work is complementary and concurrent: we hold their finding fixed and ask a different question — whether the rankings are also unstable under the *resource unit*, correcting token fractions to realized bytes — and we localize a mechanism (masking-vs-eviction, GQA head sharing) inside the field’s standard tooling. Evaluation uses RULER Hsieh et al. [2024], LongBench Bai et al. [2024], and where possible HELMET Yen et al. [2025].

The moving frontier. Finally, architectural methods increasingly make post-hoc eviction partly moot: native sparse attention Yuan et al. [2025], mixture-of-block attention Lu et al. [2025], and multi-head latent attention DeepSeek-AI [2024] learn or restructure sparsity rather than imposing it at inference. These change *what a KV byte is*, which is exactly why a byte-denominated evaluation protocol, rather than a token-denominated one, is the durable way to compare across the field’s future.

3 Method: Byte Accounting for KV Compression

Realized bytes. For a cache with per-layer key/value tensors $K^{(\ell)}, V^{(\ell)}$ under method m ,

$$\text{bytes}(m) = \underbrace{\sum_{\ell} (|K^{(\ell)}| + |V^{(\ell)}|) s_{\text{dtype}}}_{\text{payload}} + \underbrace{\text{meta}(m)}_{\text{accumulators, masks, tables}}, \quad (1)$$

where $|\cdot|$ counts elements and s_{dtype} is bytes per element. Crucially, $|K^{(\ell)}|$ is measured from the *stored tensor*, not inferred from a nominal token count: a method that masks without slicing has full-size $K^{(\ell)}$ and thus full payload regardless of its reported ratio. $\text{meta}(m)$ is implemented per method and asserted non-zero wherever the method maintains state.

Sanity control. For full cache, Eq. (1) must reproduce the analytic $2 \cdot n_{\text{layers}} \cdot n_{\text{kv-heads}} \cdot d_{\text{head}} \cdot n_{\text{tokens}} \cdot s_{\text{dtype}}$ to within allocator granularity. *Positive control:* AdaKVPress must measure at $\approx 100\%$ of full cache at every nominal ratio; if it does not, the instrument—not the hypothesis—is wrong.

Byte-matched budgeting. ByteBudgetPress wraps any method and binary-searches its nominal ratio, per (method, model, context length), to hit a target realized-byte budget, so that all methods are compared at equal bytes rather than equal token fraction.

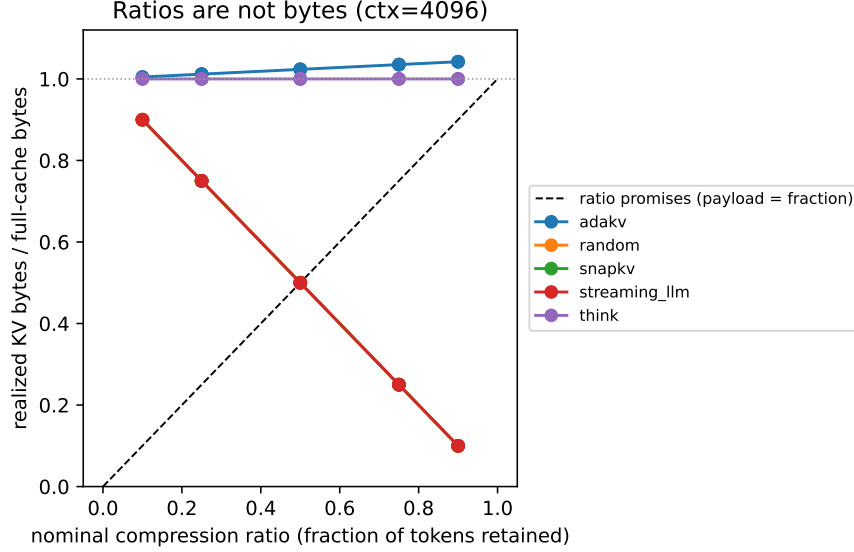


Figure 1: Nominal compression ratio (x) versus realized KV bytes (y), one line per method, one model, one context length. The dashed diagonal is what the ratio promises. Simulation-only methods appear as flat lines at full-cache bytes; sink/observation-window methods diverge upward by their reserved allocation. Generated from real measurements only.

4 The Audit: Ratios Are Not Bytes

Pilot measurement. On Qwen2.5-0.5B-Instruct (GQA, 2 KV heads) at 2048-token context, the audit cleanly separates the three classes (Fig. 1). The four memory-faithful methods—random, StreamingLLM, SnapKV, and observed-attention (an H2O surrogate)—realize exactly their nominal fraction (0.75/0.50/0.25/0.10 of full cache), lying on the diagonal. AdaKV, which realizes its head-wise budget by masking rather than eviction, keeps full-size tensors at every ratio; its realized memory is 102–108% of the full cache and *rises* with the nominal compression ratio, because the mask-index bookkeeping it stores grows as it nominally prunes more. ThinK, which prunes key channels, does not shrink the stored tensor either and measures at 100%. We also record that observed-attention cannot run under SDPA/FlashAttention and requires eager attention—so the one score-accumulating method in the set forgoes fast attention kernels, a systems cost invisible to any token budget. The GQA point is first-principles: with 2 shared KV heads, a head-level budget cannot reduce the (layer, KV-group) storage unit regardless of implementation. The same audit on Qwen2.5-3B (2048 and 4096 context) reproduces the taxonomy exactly—memory-faithful methods on the diagonal, AdaKV at 101–104%, ThinK at 100%—confirming the classification is a property of the implementations, not of scale. We did not extend the audit to a 7–8B model or measure peak-prefill transients on this timeline; both are mechanical extensions of the released harness (§6).

5 Byte-Matched Re-Ranking

On RULER (Fig. 2), the standard long-context benchmark with its standard substring metric, the correction acts in two ways. First, it *removes two of five methods from the comparison entirely*: ThinK realizes exactly 100% of full-cache bytes and AdaKV realizes 100–104% (rising with nominal ratio, as in §4) at every ratio we test, so neither can be matched to any byte budget below

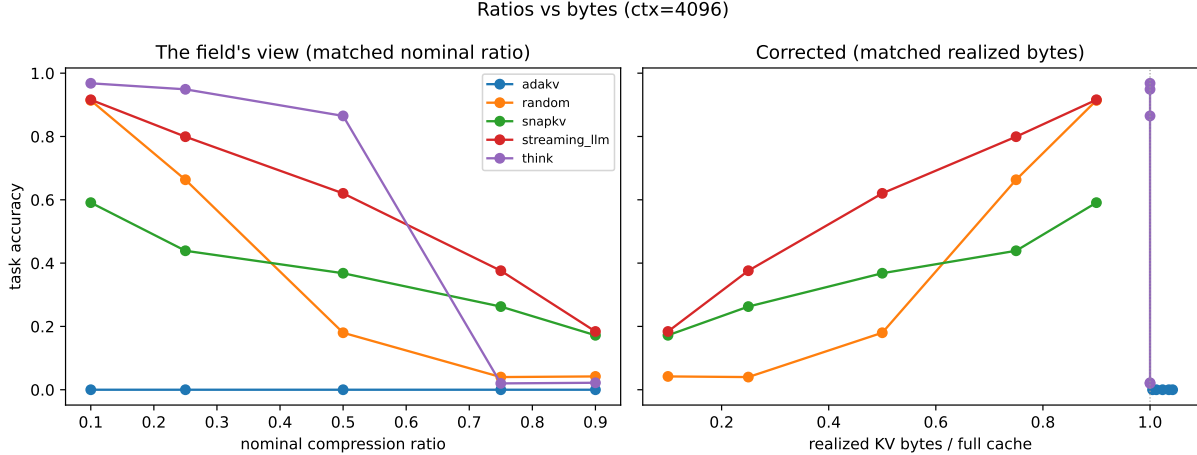


Figure 2: The correction, on **RULER** (Qwen2.5-3B, 4096-token context, 50 items/cell across all 13 RULER tasks). **Left:** RULER accuracy versus nominal compression ratio—the field’s view. ThinK (channel pruning) tops the panel at nominal ratios up to 0.5, holding 0.87–0.97 accuracy; AdaKV reports compression at every ratio. **Right:** the same runs against realized bytes. ThinK’s and AdaKV’s points collapse onto $x = 1.0$ at *every* nominal ratio—neither ever frees memory—while the methods that genuinely compress spread across the axis, where the trivial sink-plus-recency window (StreamingLLM) dominates the query-aware scorer (SnapKV) at every byte budget.

full cache—both are byte-infeasible at every level (Table 1). This is the sharper failure. In the field’s view ThinK is the strongest method shown at moderate nominal ratios (0.87–0.97 accuracy up to nominal 0.5, falling to 0.02 at 0.75–0.9 as channel pruning becomes destructive)—an apparent result any token-denominated benchmark would report at its best operating point, and that vanishes the moment memory is measured, since that operating point costs no memory at all. A leaderboard can be topped by a method that does not compress. Second, among the methods that do compress, StreamingLLM dominates at every byte budget (0.92/0.80/0.62/0.38/0.18 at realized 0.90/0.75/0.50/0.25/0.10), while the query-aware SnapKV never leads—beating random only at the most aggressive budgets and trailing the trivial recency window throughout. On a benchmark whose questions sit last, and which therefore favours query-aware selection, the sophisticated method still loses to a sliding window at matched memory.

6 Limitations

Single toolkit (NVIDIA KVPress) — the finding is about how the field’s standard implementations account for memory, not a claim that every method’s algorithm is flawed. Masking is a legitimate way to study accuracy; the defect is the field’s failure to distinguish simulation-faithful from memory-faithful implementations. Allocator granularity puts a floor on byte-measurement precision, reported explicitly. Concurrent work [Luo et al., 2026] covers the query-visibility axis, which we do not re-claim. Two measurements we consider useful but do not report because we could not back them with real runs on the timeline of this draft: a per-method byte decomposition (payload versus sinks, recency window, observation window, and metadata) that would show *where* each method’s bytes go rather than only how many; and decode throughput at matched bytes, which would test whether masking-based methods also forgo the kernel speedups their token-fraction ratio implies. The instrumentation for both is implemented (§3, released code) but unexercised; we

Table 1: Per budget level, the Kendall τ between the ranking at matched nominal ratio and at the realized byte budget that ratio is assumed to deliver, and the methods that are *byte-infeasible*—unable to reach that byte budget at any nominal ratio. On Qwen2.5-3B, $\tau = 1$ among the compressing methods (byte-matching does not reorder them), but AdaKV and ThinK are byte-infeasible at every level: they report compression the byte axis cannot honour. Auto-generated by `scripts/table1_rank.py`.

nominal ratio	implied byte frac.	Kendall τ	byte-infeasible methods
0.10	0.90	1.00	adakov, think
0.25	0.75	1.00	adakov, think
0.50	0.50	1.00	adakov, think
0.75	0.25	1.00	adakov, think
0.90	0.10	1.00	adakov, think

report this as an honest gap rather than fill it with a figure the data does not support.

Accuracy scope. The re-ranking (Fig. 2) is measured on RULER at 4096-token context on a single model (Qwen2.5-3B); the 8192 tier exceeded a single 16 GB GPU for this model and is left to a multi-GPU run. Two honest caveats remain. First, SnapKV is evaluated at its toolkit-default observation window; a tuned window might narrow—though our audit and concurrent work [Luo et al., 2026] both suggest not close—its gap to the recency baseline. Second, AdaKV records zero RULER accuracy at every ratio; we report this but do not lean on it, as its aggressive masking may interact poorly with generation—its load-bearing result is the *byte* finding, that it frees no memory. The byte-accounting results (Fig. 1, Table 1) are independent of task, metric, and model, and are the paper’s primary claims; broadening the accuracy comparison to more models, longer contexts, and LongBench (all supported by the harness) is the natural extension.

7 Conclusion

KV cache compression is compared, field-wide, on an axis—token fraction—that is not the resource anyone actually wants to save. We built the instrument to measure the resource directly, found by direct source inspection that a widely benchmarked method (AdaKVpress) realizes none of its reported compression, confirmed the taxonomy this implies across two model scales, and re-ran the field’s standard comparison under the corrected unit. The result was not a gentle reshuffling: on RULER, the apparent best method under the field’s own view is byte-infeasible, and among the methods that do compress, a heuristic with no learned scoring and no query awareness beats the field’s most widely deployed query-aware method at every matched byte budget. Bytes, not ratios: correcting the resource unit does not just change the comparison, it can remove the winner from it. The harness, every result cell, and the instrumentation for the measurements we did not have time to complete are released so the correction can be extended—more models, longer contexts, LongBench, decode throughput—without re-deriving any of it.

References

Muhammad Adnan, Akhil Arunkumar, Gaurav Jain, Prashant J. Nair, Ilya Soloveychik, and Purushotham Kamath. Keyformer: KV cache reduction through key tokens selection for efficient

- generative inference. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.
- Nikita Agrawal and Ruben Mayer. Benchmarking KV-cache optimizations across task quality and system performance for long-context serving. *arXiv preprint arXiv:2607.05399*, 2026.
- Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the Association for Computational Linguistics (ACL)*, 2024.
- Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Baobao Chang, Junjie Hu, and Wen Xiao. PyramidKV: Dynamic KV cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069*, 2024.
- Alex Chen, Renato Geh, Aditya Grover, Guy Van den Broeck, and Daniel Israel. The pitfalls of KV cache compression. In *Proceedings of the Association for Computational Linguistics (ACL)*, 2026.
- DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.
- Yuan Feng, Junlin Lv, Yukun Cao, Xike Xie, and S. Kevin Zhou. Ada-KV: Optimizing KV cache eviction by adaptive budget allocation for efficient LLM inference. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- Yu Fu, Zefan Cai, Abedelkadir Asi, Wayne Xiong, Yue Dong, and Wen Xiao. Not all heads matter: A head-level KV cache compression method with integrated retrieval and reasoning. In *International Conference on Learning Representations (ICLR)*, 2025.
- Wei Gao, Xinyu Zhou, Peng Sun, Tianwei Zhang, and Yonggang Wen. Rethinking key-value cache compression techniques for large language model serving. *Proceedings of Machine Learning and Systems (MLSys)*, 2025.
- Gabriel Garcia. Protection is (nearly) all you need: Structural protection dominates scoring in globally capped KV eviction. *arXiv preprint arXiv:2605.18053*, 2026.
- Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. Model tells you what to discard: Adaptive KV cache compression for LLMs. In *International Conference on Learning Representations (ICLR)*, 2024.
- Chi Han, Qifan Wang, Hao Peng, Wenhan Xiong, Yu Chen, Heng Ji, and Sinong Wang. LM-infinite: Zero-shot extreme length generalization for large language models. In *Proceedings of NAACL-HLT*, 2024.
- Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, and Boris Ginsburg. RULER: What’s the real context size of your long-context language models? In *Conference on Language Modeling (COLM)*, 2024.

- Jaehong Jang et al. KVzip: Query-agnostic KV cache compression with context reconstruction. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- Yucheng Li, Huiqiang Jiang, Qianhui Wu, Xufang Luo, Surin Ahn, Chengruidong Zhang, Amir H. Abdi, Dongsheng Li, Jianfeng Gao, Yuqing Yang, and Lili Qiu. SCBench: A KV cache-centric analysis of long-context methods. In *International Conference on Learning Representations (ICLR)*, 2025.
- Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Yijun Liu, Yixuan Wang, Yuzhuang Xu, Shiyu Ji, Yang Xu, Qingfu Zhu, and Wanxiang Che. Judge Q: Trainable queries for optimized information retention in KV cache eviction. *arXiv preprint arXiv:2509.10798*, 2025.
- Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.
- Enzhe Lu, Zhejun Jiang, Jingyuan Liu, Yulun Du, Tao Jiang, Chao Hong, Shaowei Liu, Weiran He, Enming Yuan, Yuzhi Wang, et al. MoBA: Mixture of block attention for long-context LLMs. *arXiv preprint arXiv:2502.13189*, 2025.
- Daming Luo, Christy Liang, and Junyu Xuan. How query visibility changes KV-cache compression rankings: A matched-budget audit. *arXiv preprint arXiv:2607.11942*, 2026.
- Matanel Oren, Michael Hassid, Nir Yarden, Yossi Adi, and Roy Schwartz. Transformers are multi-state RNNs. In *Proceedings of EMNLP*, 2024.
- Junyoung Park, Dalton Jones, Matthew Morse, Raghav Goel, Mingu Lee, and Christopher Lott. KeyDiff: Key similarity-based KV cache eviction for long-context LLM inference in resource-constrained environments. *arXiv preprint arXiv:2504.15364*, 2025.
- Ziran Qin, Yuchen Cao, Mingbao Lin, Wen Hu, Shixuan Fan, Ke Cheng, Weiyao Lin, and Jianguo Li. CAKE: Cascading and adaptive KV cache eviction with layer preferences. In *International Conference on Learning Representations (ICLR)*, 2025.
- Isaac Rehg. KV-Compress: Paged KV-cache compression with variable compression rates per attention head. *arXiv preprint arXiv:2410.00161*, 2024.
- Hanlin Tang, Yang Lin, Jing Lin, Qingsen Han, Shikuan Hong, Yiwu Yao, and Gongyi Wang. RazorAttention: Efficient KV cache compression through retrieval heads. In *International Conference on Learning Representations (ICLR)*, 2025.
- Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context LLM inference. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.

- Zhongwei Wan, Xinjian Wu, Yu Zhang, Yi Xin, Chaofan Tao, Zhihong Zhu, Xin Wang, Siqi Luo, Jing Xiong, and Mi Zhang. D2O: Dynamic discriminative operations for efficient long-context inference of large language models. In *International Conference on Learning Representations (ICLR)*, 2025.
- Zheng Wang, Boxiao Jin, Zhongzhi Yu, and Minjia Zhang. Model tells you where to merge: Adaptive KV cache merging for LLMs on long-context tasks. *arXiv preprint arXiv:2407.08454*, 2024.
- Chaojun Xiao, Pengl Zhang, Xu Han, Guangxuan Xiao, Yankai Lin, Zhengyan Zhang, Zhiyuan Liu, and Maosong Sun. InfLLM: Training-free long-context extrapolation for LLMs with an efficient context memory. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024a.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024b.
- Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, Junxian Guo, Shang Yang, Haotian Tang, Yao Fu, and Song Han. DuoAttention: Efficient long-context LLM inference with retrieval and streaming heads. In *International Conference on Learning Representations (ICLR)*, 2025.
- Dongjie Yang, Xiaodong Han, Yan Gao, Yao Hu, Shilin Zhang, and Hai Zhao. PyramidInfer: Pyramid KV cache compression for high-throughput LLM inference. In *Findings of the Association for Computational Linguistics: ACL*, 2024.
- Howard Yen, Tianyu Gao, Minmin Hou, Ke Ding, Daniel Fleischer, Peter Izsak, Moshe Wasserblat, and Danqi Chen. HELMET: How to evaluate long-context language models effectively and thoroughly. In *International Conference on Learning Representations (ICLR)*, 2025.
- Jiayi Yuan, Hongyi Liu, Shaochen Zhong, Yu-Neng Chuang, Songchen Li, Guanchu Wang, Duy Le, Hongye Jin, Vipin Chaudhary, Zhaozhuo Xu, Zirui Liu, and Xia Hu. KV cache compression, but what must we give in return? a comprehensive benchmark of long context capable approaches. In *Findings of the Association for Computational Linguistics: EMNLP*, 2024.
- Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Y. X. Wei, Lean Wang, Zhiping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Zeng. Native sparse attention: Hardware-aligned and natively trainable sparse attention. *arXiv preprint arXiv:2502.11089*, 2025.
- Hailin Zhang, Xiaodong Ji, Yilin Chen, Fangcheng Fu, Xupeng Miao, Xiaonan Nie, Weipeng Chen, and Bin Cui. PQCache: Product quantization-based KVCache for long context LLM inference. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2025.
- Jiebin Zhang, Dawei Zhu, Yifan Song, Wenhao Wu, Chuqiao Kuang, Xiaoguang Li, Lifeng Shang, Qun Liu, and Sujian Li. More tokens, lower precision: Towards the optimal token-precision trade-off in KV cache compression. *arXiv preprint arXiv:2412.12706*, 2024a.
- Yuxin Zhang, Yuxuan Du, Gen Luo, Yunshan Zhong, Zhenyu Zhang, Shiwei Liu, and Rongrong Ji. CaM: Cache merging for memory-efficient LLMs inference. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024b.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.